

Repositorio: github.com/LeonardoAmundarain/reposicion-stock-agent-lamundarain_fespinoza

1. Resumen ejecutivo

Construimos un agente que decide qué reponer, qué devolver y qué comprar para dos tiendas y el canal online de una perfumería, de cara a la temporada alta de noviembre y diciembre. Trabaja con 5 meses de ventas reales (44.393 líneas) y el stock de 11.712 productos.

El agente combina 7 herramientas de consulta, razonamiento y escritura, memoria en tres niveles y un planner de 7 reglas con prioridades y replanificación. Conversa con Gemini o Claude mediante LangChain, pero ningún pedido se registra sin pasar por las reglas.

Con fecha de plan 3 de noviembre de 2026 propone reponer 2.839 unidades, devolver 4.978 a la bodega central y comprar 5.206 unidades de 445 productos. Las 91 pruebas automáticas pasan, y el proyecto se instala y abre con un iniciador.

2. Contexto organizacional y requerimiento

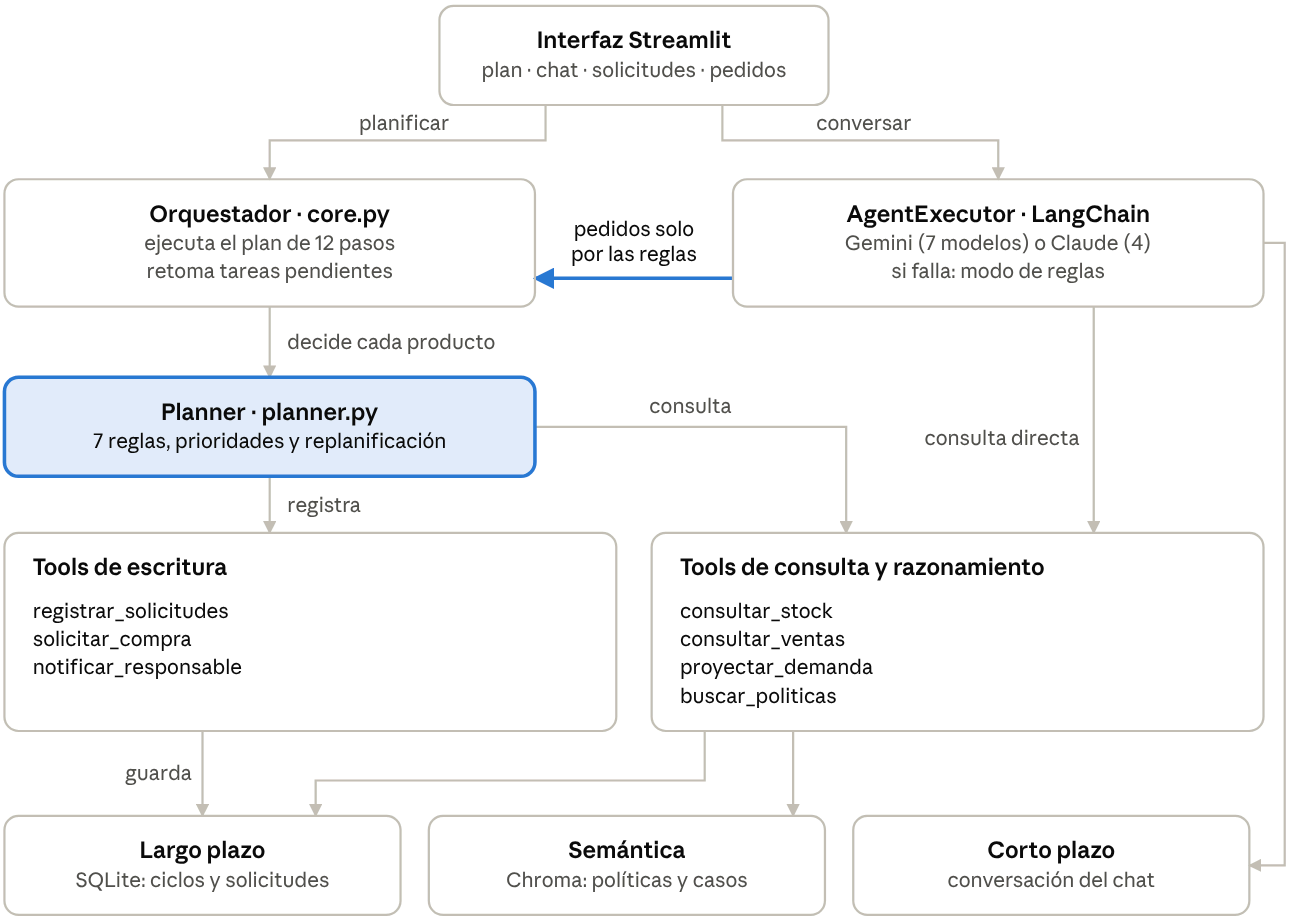
La empresa abastece sus locales desde una bodega central y vende miles de productos, por lo que decidir a mano qué pedir obliga a cruzar ventas con stock producto por producto. El requerimiento es automatizar esa decisión para que cada local tenga stock suficiente en temporada alta, sin acumular lo que no se vende.

Local	Línea	Vende por	Reposición
14	Tienda diseñador	Caja 14	Hasta 2 veces por semana, 2 días de demora
15	Tienda árabes	Caja 15	Hasta 2 veces por semana, 2 días de demora
16	Online, todas las líneas	Caja 37	2 veces al mes, por quincena

El proyecto continúa la EP1, donde un agente consultaba ventas y rentabilidad ya calculadas y las explicaba en lenguaje natural. En la EP2 el agente además razona con reglas del negocio, planifica y registra acciones. El alcance excluye mover mercadería o emitir órdenes de compra reales: todo queda como solicitud que una persona confirma.

3. Arquitectura y orquestación

Ningún pedido se registra sin pasar por las reglas



arquitectura · interfaz, orquestador, planner, 7 tools y 3 memorias

El chat consulta directamente, pero cualquier pedido, venga del botón o del LLM, pasa por el orquestador y las 7 reglas del planner antes de escribir en SQLite. El código usa LangChain 1.x con `langchain-classic` (AgentExecutor), Chroma, SQLite y Streamlit.

4. Herramientas del agente

El agente usa 7 herramientas definidas con `@tool` de LangChain: cuatro consultan o razonan y tres escriben. El LLM solo accede directamente a las de consulta; para escribir usa `planificar_reposicion` y `evaluar_pedido_tienda`, que siempre pasan por las reglas.

Herramienta	Tipo	Qué hace
`consultar_stock`	Consulta	Stock por producto en cada bodega y en la central
`consultar_ventas`	Consulta	Unidades vendidas y venta diaria por local
`proyectar_demanda`	Razonamiento	Venta esperada, con factor 1,5 en temporada alta
`buscar_politicas`	Consulta semántica	Políticas y casos anteriores por significado
`registrar_solicitudes`	Escritura	Reposiciones y devoluciones; revisa la central antes de guardar
`solicitar_compra`	Escritura	Compras pendientes de confirmación
`notificar_responsable`	Escritura simulada	Aviso al local o a compras

5. Memoria y recuperación de contexto

La memoria de largo plazo es la que da continuidad: el agente sabe qué viene en camino, no repite pedidos y retoma una tarea cuando llega una compra, incluso después de reiniciar la aplicación.

Nivel	Dónde	Qué permite
Corto plazo	`ConversationBufferMemory`	Recordar el local mencionado: «¿y cuántas solicitudes quedaron en esa tienda?»
Largo plazo	SQLite: tablas `ciclos` y `solicitudes`	Restar lo que viene en camino, respetar la frecuencia de pedidos y retomar productos al recibir una compra
Semántica	Chroma con embeddings multilingües	Encontrar la política que respalda cada acción y recordar ciclos anteriores

La política se divide en 28 fragmentos, uno por regla. En una prueba de 17 preguntas redactadas con otras palabras, el modelo multilingüe encontró la sección correcta en primer lugar 12 veces; el modelo en inglés de la guía, 6 veces con fragmentos por sección.

Como acierta 12 de 17, las decisiones no dependen de esta búsqueda: solo la usan para citar la fuente.

6. Planificación y toma de decisiones

Cada producto pasa por 7 reglas en orden, y la primera que se cumple decide:

1. Tienda con stock y sin ventas en 60 días: devolver a la bodega central.
2. Producto de la línea de otra tienda: no se repone y se vende hasta agotarse.
3. El local no lo vende: no se pide.
4. Stock más lo que viene en camino cubre el objetivo: no se pide.
5. La central cubre la necesidad: reponer la diferencia.
6. La central cubre una parte: reponer lo que hay y comprar el resto.
7. La central no tiene unidades: solicitar compra.

El objetivo de una tienda cubre 7 días de venta más 2 de demora; el de online, la venta hasta fin de quincena. Un día entre el 16 de noviembre y el 21 de diciembre vale 1,5 días normales.

Cuando la central no alcanza para todos, el agente atiende primero lo que está sin stock, luego lo que se agota antes de que llegue la reposición y, dentro de cada grupo, lo que más vende. El plan de 12 pasos se adapta en dos momentos: quita los pasos sin trabajo y, si la central cambia mientras registra, reduce la cantidad y agrega la compra del faltante.

7. Resultados y evidencias

Con los datos reales y fecha de plan 3 de noviembre de 2026, el agente propone lo siguiente; las tiendas se planifican antes que online porque su necesidad es de la semana.

Local	Reponer (unidades)	Devolver a central (unidades)	Productos de otra línea
15 Árabes	403	2.408	470
14 Diseñador	114	2.570	561
16 Online	2.322	No aplica	No aplica

La solicitud de compra suma 5.206 unidades de 445 productos. Al revisar el resultado contra los archivos originales, ningún producto con venta reciente quedó para devolver y la bodega central no quedó sobreasignada.

Evidencia en el repositorio:

- **Pruebas:** 91 pruebas automáticas pasan, también en una instalación limpia (`evidence/pytest_output.txt`).
- **Escenarios:** 11 casos de decisión: quiebre, devolución, compra, memoria, temporada alta, replanificación, pedidos de tienda, retomar una tarea y frecuencia online (`evidence/escenarios.json`).
- **LLM real:** conversaciones con Gemini y Claude en las que el agente elige sus herramientas y rechaza una orden de saltarse las reglas sin registrar nada (`evidence/conversacion_gemini.json`, `evidence/conversacion_claude.json`).
- **Respaldo de Gemini:** con 3.8 Flash saturado, el agente respondió con 3.6 Flash y lo avisó bajo la respuesta.
- **Pronóstico:** frente a las ventas reales de octubre, el promedio del período se desvió cerca de 10% en las tiendas y 12% en online.

8. Limitaciones

- El pronóstico por producto es poco confiable, con errores cercanos al 100%; el total por local sí se acerca a lo real.
- No hay historial de quiebres: una venta en cero por falta de stock parece falta de demanda.
- El factor 1,5 y sus fechas son una estimación del negocio, no un cálculo con años anteriores.
- La venta que migra de una tienda a la otra aún no se suma al objetivo de la tienda que la recibe.
- El agente no descuenta lo vendido entre un pedido y el siguiente: hay que cargar un archivo de stock actualizado.
- La lista de compra puede incluir productos descontinuados; una persona debe revisarla.
- En el nivel gratuito, Gemini puede saturarse, y la redacción de cualquier LLM varía entre respuestas.

9. Justificación de las decisiones de diseño

- **Framework y LLM:** Elegimos LangChain (AgentExecutor) integrado con Gemini como opción nativa gratuita (3.6 Flash/3.5 Flash-Lite) y Claude como alternativa avanzada de pago, asegurando flexibilidad en los costos de ejecución y respuestas en lenguaje natural contextualizadas para los usuarios.
- **Separación entre reglas y LLM:** Implementamos las reglas de negocio en un componente independiente (planner.py), que core.py usa como modo de reglas, para que el negocio siga operando con lógica dura determinista incluso si no hay conexión a los LLM o fallan las API keys, garantizando que la IA actúe como un facilitador de interfaz y no como una caja negra crítica.
- **Memoria (corto plazo, largo plazo, semántica):** Diseñamos una estrategia triple utilizando ConversationBufferMemory para retener el contexto inmediato del local, registros en SQLite para la persistencia del estado ante reinicios (saber qué pedidos vienen en camino), y Chroma DB para recuperar políticas organizacionales de manera semántica.
- **Planificación y prioridades:** El componente planner.py define un plan adaptativo de 12 pasos, que el orquestador core.py ejecuta, y jerarquiza los productos por nivel de urgencia (por ejemplo, quiebres de stock), optimizando la distribución de la bodega central y recalculando dinámicamente las cantidades requeridas según los objetivos de temporada alta.
- **Almacenamiento:** Adoptamos un enfoque híbrido local con SQLite para estructurar tablas relacionales de ciclos y solicitudes (datos de fácil auditoría) junto con Chroma DB como almacenamiento vectorial para el indexado eficiente de documentos informativos y casos previos.
- **Interfaz:** Desarrollamos una aplicación web interactiva en Streamlit que unifica de forma simple tres flujos clave para el negocio: la visualización del plan de reposición automático, una ventana de chat directo con el agente de IA y un formulario que evalúa con las reglas los pedidos que envían las tiendas.

10. Conclusiones y reflexiones individuales

Conclusiones del equipo:

El desarrollo de este agente funcional demostró que la automatización de la cadena de suministro en un entorno con miles de SKUs reales requiere necesariamente un enfoque híbrido. La integración de LangChain con una base lógica tradicional nos permitió construir una solución capaz de procesar de manera masiva y consistente la data de la

perfumería. Logramos comprobar que un agente de IA es sumamente eficiente como interfaz de usuario y optimizador de flujos comerciales (proponiendo el balanceo idóneo de stock y devoluciones para la temporada alta), siempre y cuando sus acciones de escritura estén estrictamente gobernadas por un motor de reglas determinista. Esto anula el riesgo de alucinaciones financieras o logísticas y garantiza la total auditabilidad del sistema ante el negocio.

Reflexión de Leonardo Luis Amundarain Cabrera (aprendizaje y contribución):

Mi mayor aprendizaje en este proyecto estuvo en el diseño y la orquestación técnica de una arquitectura de software multi-modelo y su sistema de persistencia. El gran desafío inicial fue buscar información y documentación técnica actualizada sobre marcos de agentes en constante evolución como LangChain 1.x, lo que nos obligó a profundizar en arquitecturas robustas para conectar herramientas operativas de consulta y escritura de forma segura. Contribuí principalmente en el desarrollo de la lógica del orquestador central (core.py), la implementación de la memoria a largo plazo en SQLite y la integración semántica de Chroma. Uno de los hitos más complejos y enriquecedores fue evaluar distintas IA de forma analítica: comparamos empíricamente modelos de embeddings multilingües frente a modelos tradicionales en inglés, descubriendo que la contextualización local de las políticas del negocio requería una selección meticulosa del modelo para elevar el acierto de recuperación en las pruebas semánticas.

Reflexión de Francisco Miguel Espinoza Moyano (aprendizaje y contribución):

Para mí, el proceso representó un profundo viaje de aprendizaje en la traducción de restricciones comerciales abstractas a reglas de planificación dinámicas que la IA pudiera seguir fielmente. El principal desafío operativo consistió en investigar y estructurar cómo mitigar las limitaciones intrínsecas de los LLM (como la variabilidad en la redacción o la saturación de los proveedores) mediante el diseño de contingencias a nivel de código. Mi contribución se centró en la creación y el testeo del módulo de prioridades y reglas adaptativas de 12 pasos dentro de planner.py, asegurando que las decisiones respondieran correctamente a variables externas como el factor de temporada alta. El proceso de probar y contrastar las capacidades conversacionales de Gemini y Claude bajo condiciones de estrés nos enseñó que la resiliencia de una solución de IA no depende de un único proveedor, sino de nuestra capacidad para diseñar sistemas capaces de conmutar de forma transparente ante fallas de API, protegiendo siempre la continuidad de la operación comercial.

11. Declaración de uso de IA

Herramienta	Uso	Validado por
Gemini	Motor de conversación del agente (opción por defecto)	Leonardo Amundarain y Francisco Espinoza
Claude (API de Anthropic)	Motor de conversación del agente (opción seleccionable)	Leonardo Amundarain
Claude Code (Anthropic, Claude Opus, Hikun, Fable, Sonet)	Generación de parte del código, datos simulados, diagrama mermaid	Leonardo Amundarain y Francisco Espinoza